

WebRTC + Android 개발 자료조사

사용 기술

- 백엔드
 - node.js
 - mediasoup 라이브러리
 - SFU 라이브러리 (Selective Forwarding Unit)
 - 중앙에 SFU 서버를 둔다
 - 각 클라이언트가 자신의 미디어를 서버에 업로드한다
 - 서버가 해당 미디어가 필요한 클라이언트한테만 전달한다.
 - 각 클라이언트가 단 하나의 업로드 스트림만 보내면 된다.
- 1. 시그널링 서버 (Node.js, Express.js/Socket.IO)
 - 사용자 인증, 룸 생성/관리, 권한 제어
 - 각 사용자는 Socket.IO를 통해 “참여”, “나가기” 등 이벤트를 처리
 - 실제 오디오/비디오 데이터는 시그널링 서버에서 직접 중계하지 않고 mediasoup에 전달
- 2. mediasoup SFU 서버
 - Node.js 내에 mediasoup을 통합하거나, 별도의 프로세스로 동작 가능
 - 각 사용자의 RTP/RTCP 스트림을 받고, 다른 사용자에게 필요한 형태로 전달
- 3. 클라이언트(안드로이드 + Kotlin)
 - WebRTC Native 라이브러리(구글에서 제공하는 .aar)를 사용
 - 시그널링 서버와 통신하여 SFU에 접속하기 위한 매개변수(트랜SPORT 정보, ICE 서버 정보 등)를 공유
 - 미디어 스트림(오디오/비디오)을 로컬에서 캡처 → SFU로 전송 → SFU가 다른 클라이언트들에게 중계

구현 순서 개요

1. mediasoup 서버 셋업
 - Node.js 프로젝트에서 `npm install mediasoup`
 - mediasoup Worker를 생성하고, 각 화상 통화 방(Room)에 맞게 Router, Transport 등을 구성
 - CPU 코어별 Worker 분산 등 고성능을 위한 설정
2. 시그널링 서버
 - Express.js나 Fastify, Socket.IO 등을 사용
 - 클라이언트가 접속하면 “방 만들기” / “방 입장” / “나가기” 등 이벤트 처리
 - mediasoup Router/Transport를 연결하고, 클라이언트에게 필요한 정보를 전달
 - 예: “이 Transport를 통해 오디오/비디오를 publish하세요”
3. TURN/STUN 서버
 - P2P 직결이 힘든 상황에서도 SFU에 접속 가능하도록 TURN/STUN 서버 설정
 - coturn을 사용하여 `turn:yourdomain.com` + 계정/비밀번호 세팅
4. 안드로이드 앱 (Kotlin + WebRTC 라이브러리)
 - 시그널링: Socket.IO 클라이언트 / WebSocket 사용
 - **PeerConnectionFactory** 등 WebRTC 초기화

- mediasoup을 쓸 경우, 전통적인 PeerConnection 흐름과는 다르게 SFU 방식으로 데이터를 보냄/받음
 - mediasoup 클라이언트 예제(안드로이드)는 직접 구현하거나, 참고 라이브러리를 찾아야 함(일반 WebRTC와 비슷하지만, 소켓 이벤트 및 SDP 교환 로직이 조금 다름)
- 카메라/마이크 권한 → 로컬 스트림 생성 → SFU에 Publish → 다른 사용자 스트림 Subscribe

5. 테스트 & 배포

- 그룹 통화 성능 테스트(동시 접속자 수)
- Docker, Kubernetes 등의 컨테이너 환경에서 확장 가능성 고려
- HTTPS 적용(안드로이드 앱이면 직접 RTCPeerConnection 설정 시 HTTPS가 필수는 아니지만, 보안을 위해 TLS를 쓰는 것이 권장)

중요한 포인트 & 팁

1. 방 개념(Rooms)

- 시그널링 서버에서 “방” 단위로 사용자들을 관리
- 각 방마다 mediasoup Router를 매핑하거나, 혹은 하나의 Router에 여러 “방”을 구현하는 구조 선택

2. 오디오/비디오 품질 관리

- SFU 기반에서는 각 사용자마다 해상도, 비트레이트를 다르게 Subscribe할 수 있음(“Simulcast” 기능)
- 네트워크 상태가 안 좋은 사용자는 저화질 스트림으로 전환

3. 확장성

- **mediasoup**: 여러 워커 프로세스를 생성하여 CPU 코어를 활용
- 트래픽이 더 늘어나면, 여러 대 서버에 로드 밸런싱

4. UI/UX (안드로이드 앱)

- 참여자 비디오 타일(grid) 구조
- 스피커 감지(활성 화자 감지)
- 화면 공유(스크린 캡처) 기능을 고려한다면 안드로이드 권한 처리와 별도의 트랙 생성 로직 필요

5. 보안

- 미디어 자체는 SRTP(WebRTC 자체 암호화)로 보호되지만, 인증/인가(누가 어느 방에 들어갈 수 있는지) 로직은 별도로 구현해야 함
- JWT나 OAuth 등을 통해 사용자를 인증 가능

6. 오픈소스 솔루션 사용 여부

- 만약 직접 구현이 부담된다면, **Jitsi**, **OpenVidu**, **Daily.co**, **Twilio** 등 완성도 높은 솔루션을 활용하는 것도 방법
- 커스텀 기능이 많으면 mediasoup 같은 프레임워크를 직접 다루는 편이 유연

결론 & 요약

- 그룹 화상 통화를 Node.js 기반으로 구현하려면, 일반 P2P 방식 대신 **SFU** 또는 **MCU** 서버를 도입해야 확장성과 성능을 확보할 수 있습니다.
- Node.js 환경에서 대표적으로 많이 쓰이는 **mediasoup**이 강력하고 유연한 SFU 라이브러리이므로, 시그널링 서버 (Express.js/Socket.IO)와 결합해 그룹 화상회의 기능을 구현할 수 있습니다.
- 안드로이드 앱(Kotlin)에서는 Google WebRTC 라이브러리를 사용하되, SFU와의 “시그널링 절차”를 맞춰서 SDP 교환, ICE 설정 등을 다뤄야 합니다.
- 추가적으로 **TURN/STUN** 서버, 인증/인가, 방(룸) 관리, **UI/UX** 설계 등도 종합적으로 고려해 프로젝트를 진행하면 됩니다.

구현 과정?

- **Node.js + mediasoup**

- Node.js 서버에서 **mediasoup** 라이브러리를 사용하여 SFU를 구성합니다.
 - 시그널링을 처리할 **Socket.IO**(또는 다른 WebSocket) 서버를 마련하고, 클라이언트들 간의 방(룸), 접속/퇴장, SDP, ICE 등 필요한 정보를 교환하는 로직을 구현합니다.
 - mediasoup을 통해 오디오/비디오 트랜스포트(Transport)**를 생성하고, 각 클라이언트가 Publish(송출) / Subscribe**(구독)하도록 설정합니다.
- **안드로이드 (Kotlin) + WebRTC 라이브러리**
 - 구글의 WebRTC 라이브러리(.aar)를 사용하여, 로컬 미디어(카메라/마이크) 스트림을 생성합니다.
 - 시그널링 서버와 **Socket.IO** 등을 통해 연결하여, 서버에서 전달해주는 SFU(mediasoup)와의 세션 정보를 받아 SDP, ICE candidate 등을 교환합니다.
 - Publish할 트랙(오디오, 비디오)을 mediasoup에 전송하고, 다른 참여자의 트랙도 Subscribe하여 수신/표시합니다.
- **TURN/STUN 서버 (권장)**
 - NAT 뒤나 방화벽 환경에서도 안정적으로 연결하기 위해 **coturn** 같은 TURN/STUN 서버를 세팅합니다.
 - 안드로이드 클라이언트와 mediasoup에서 공용 TURN/STUN 서버 정보를 이용하도록 설정합니다.
- **기타 고려사항**
 - **방(룸) 관리**: 여러 참가자가 특정 룸에 입장하면, 해당 룸에 대한 mediasoup Router/Transport를 생성하고, 각각을 연결해줘야 합니다.
 - **인증/인가**: JWT나 OAuth 같은 방법으로 사용자 인증/권한 관리를 할 수 있습니다.
 - **UX/UI**: 안드로이드에서 참여자들의 비디오 화면을 목록/grid로 보여주는 UI를 구현하셔야 합니다. 다인 통화에서 화면 전환(누가 말할 때 메인 화면 표시) 등을 추가로 개발할 수 있습니다.
 - **배포 및 확장성**: 서비스 규모가 커진다면, Docker나 Kubernetes를 통해 **mediasoup Worker**를 확장하고, 여러 서버에 로드밸런싱을 적용할 수 있습니다.